/

- CryptoLens AI - Netlify Serverless Function
- Securely handles requests to the Google Gemini API. */

exports.handler = async (event, context) => { const headers = { 'Access-Control-Allow-Origin': '*', 'Access-Control-Allow-Headers': 'Content-Type', 'Access-Control-Allow-Methods': 'POST, OPTIONS', 'Content-Type': 'application/json' };
if (event.httpMethod === 'OPTIONS') { return { statusCode: 204, headers, body: '' }; }
if (event.httpMethod !== 'POST') { return { statusCode: 405, headers, body: JSON.stringify({ error: 'Method Not Allowed. Use POST.' }) }; }
let requestData; try { if (!event.body) { return { statusCode: 400, headers, body: JSON.stringify({ error: 'Request body is missing.' }) }; } requestData = JSON.parse(event.body); } catch (error) { return { statusCode: 400, headers, body: JSON.stringify({ error: 'Invalid JSON payload.' }) }; }
const query = requestData.query || requestData.prompt;
if (!query || typeof query !== 'string' || query.trim() === '') { return { statusCode: 400, headers, body: JSON.stringify({ error: 'Query parameter is required and cannot be empty.' }) }; }
if (query.length > 2000) { return { statusCode: 400, headers, body: JSON.stringify({ error: 'Query exceeds maximum allowed length of 2000 characters.' }) }; }
const apiKey = process.env.GEMINI_API_KEY; if (!apiKey) { console.error('CRITICAL: GEMINI_API_KEY is not set in Netlify environment variables.'); return { statusCode: 500, headers, body: JSON.stringify({ error: 'Server configuration error. Please contact support.' }) }; }
const modelName = 'gemini-2.5-flash'; const endpoint = https://generativelanguage.googleapis.com/v1beta/models/$modelName:generateContent?key=${apiKey};
const systemInstruction = "You are CryptoLens AI, an advanced analytical research assistant for cryptocurrency markets. " + "Strict Rules: Never fabricate real-time prices, market caps, trading volumes, or financial statistics. " + "Clearly distinguish AI-generated qualitative research and technical analysis from verified live market data. " + "Never execute trades, access exchange accounts, or request private credentials. " + "Maintain a professional, objective tone.";
const payload = { contents: [{ role: 'user', parts: [{ text: ${systemInstruction}\n\nUser Research Query:\n${query} }] }], generationConfig: { temperature: 0.7, maxOutputTokens: 1024 } };
const controller = new AbortController(); const timeoutId = setTimeout(() => controller.abort(), 15000);
try { const apiResponse = await fetch(endpoint, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(payload), signal: controller.signal });
clearTimeout(timeoutId);
if (!apiResponse.ok) { const errorText = await apiResponse.text(); console.error(Gemini API returned status ${apiResponse.status}:, errorText);
let userErrorMessage = 'Failed to generate AI response.'; if (apiResponse.status === 429) { userErrorMessage = 'Rate limit exceeded. Please try again later.'; } else if (apiResponse.status === 403 || apiResponse.status === 401) { userErrorMessage = 'Authentication error with AI provider.'; }
return { statusCode: apiResponse.status, headers, body: JSON.stringify({ error: userErrorMessage }) }; }
const result = await apiResponse.json(); const candidateText = result.candidates?.[0]?.content?.parts?.[0]?.text;
if (!candidateText) { return { statusCode: 502, headers, body: JSON.stringify({ error: 'Received malformed response structure from Gemini API.' }) }; }
return { statusCode: 200, headers, body: JSON.stringify({ success: true, response:

```
candidateText, disclaimer: "AI-generated analysis. Not financial advice." }) };
} catch (error) { clearTimeout(timeoutId);
if (error.name === 'AbortError') { console.error('Gemini API request timed out.'); return {
statusCode: 504, headers, body: JSON.stringify({ error: 'Gateway Timeout: The AI model took
too long to respond.' }) }; }
console.error('Internal server error during Gemini fetch:', error); return { statusCode: 500,
headers, body: JSON.stringify({ error: 'An unexpected internal error occurred while processing
your request.' }) }; } };
```